

Validate Before You Apply: An LLM-Augmented Learning Assistant with Action Validation for Computer Networks Education

Anonymous Author(s)

Submission to IEEE TALE 2026 — omitted for double-blind review

Abstract—Large language models (LLMs) promise adaptive tutoring, but when a tutoring agent can also act on the learner’s working artifact, hallucination acquires a new cost: an incorrect action silently corrupts the very object the student is reasoning about. We present CAPA8, a browser-based learning environment for computer networks education that couples a conversational LLM tutor with an interactive network topology simulator the tutor can read and modify. Its central design contribution is a pedagogical reliability layer: every model-proposed modification is expressed in a closed action grammar, parsed, and validated against the live topology state before it is applied, with ambiguous requests resolved by rule-based clarification dialogues that bypass the LLM entirely. Adaptive scaffolding is provided by a Level $\times$ Focus mode matrix that reconfigures the tutor from step-by-step guidance with analogies to a technical peer. A preliminary technical evaluation combines 47 unit tests covering the validation pipeline with a reliability benchmark of 180 model interactions across three deployment scenarios—a small and a larger local open-weight model and a cloud-hosted high-capacity model—plus an ablation of the prompt-side mechanisms, showing that the validation layer intercepts all structurally invalid actions across every model while preserving high task completion. A classroom study is planned as future work.

Index Terms—large language models, intelligent tutoring systems, scaffolding, computer networks education, hallucination mitigation, human-AI collaboration

I. INTRODUCTION

Computer networking is taught through abstractions—addressing, subnetting, routing, gateways—whose consequences only become visible when a concrete topology misbehaves. Professional simulators such as Cisco Packet Tracer [12] and emulation environments such as Mininet [13] give students that concreteness, but they carry steep learning curves, and none of them explains *why* the ping the student just issued never arrived. The explanatory gap is usually filled by an instructor, who does not scale, or lately by a general-purpose chatbot, which cannot see the student’s diagram and therefore reasons about a network that does not exist.

LLMs have shown clear potential as adaptive explainers in engineering education [1], [2]. The obvious next step—letting the tutor not only explain the student’s artifact but also act on it—introduces a reliability problem that is qualitatively different from textual hallucination [4]. A fabricated sentence can be challenged; a fabricated *action* (connecting a PC to a router that does not exist, duplicating an IP address, “fixing” a missing gateway by adding a spurious cable) rewrites the learning artifact itself. Novices are the least equipped to detect

such corruption: they cannot tell whether the inconsistency they now observe is their own misconception or the tool’s error, which undermines both trust and the diagnostic reasoning the exercise was meant to train.

CAPA8 is a no-install, browser-based environment that addresses this problem by treating the LLM as an *untrusted proposer*. The model never mutates the topology directly. It emits proposals in a closed action grammar; a deterministic pipeline classifies the student’s intent before any model call, constrains the prompt, parses the response, validates every proposed action against the live graph state, and routes batches through a human-confirmable preview with single-step undo. The same pipeline carries the pedagogy: a Level $\times$ Focus mode matrix adapts the depth and register of explanations, and a topology analyzer injects the same misconfiguration warnings the student sees on the canvas into the model’s context, so tutor and student literally look at the same network.

This paper makes three contributions: (1) the design of a validate-before-apply action pipeline as a pedagogical reliability mechanism for LLM tutors that operate on learner artifacts; (2) an adaptive scaffolding scheme that operationalizes tutoring styles as a selectable Level $\times$ Focus matrix combined with intent-dependent decoding temperature; and (3) a preliminary technical evaluation—formal unit verification plus a reliability benchmark spanning three deployment scenarios (small local, larger local, and cloud models) with an ablation of the prompt-side mechanisms—showing that validation intercepts every structurally invalid action regardless of model capability, and quantifying how much each mechanism contributes.

II. RELATED WORK

LLMs in computing education: Kasneci et al. [1] map the opportunities and risks of LLMs across education, identifying hallucination and overreliance as core concerns. Within computing education, Denny et al. [2] survey the post-GPT landscape, and CodeHelp [3] demonstrates guardrails that keep an LLM helper from emitting full solutions in programming classes. CodeHelp’s guardrails shape the *text* a model returns; CAPA8 extends the guardrail idea to *state-changing actions*, where the failure mode is not an unwanted answer but a corrupted artifact.

Scaffolding and intelligent tutoring systems: The tutor configurations in CAPA8 instantiate classic scaffolding theory: graduated assistance within the learner’s zone of proximal

development [5], [6], software realizations of structuring and problematizing supports [7], and the ITS tradition of step-based adaptive dialogue [8]. CAPA8 differs from end-to-end LLM tutors in deliberately keeping parts of the dialogue deterministic: ambiguity resolution and misconfiguration detection are rule-based, reserving the LLM for explanation and proposal generation.

Constraining LLM output: Hallucination surveys [4] motivate a growing toolbox of output constraints, from programmable conversational rails [9] to constrained semantic decoding for code [10]. These techniques operate at generation time and are stateless with respect to the application. CAPA8 adds a complementary, state-dependent stage: semantic validation of each proposed action against the live topology (referential integrity, duplicate addresses, type normalization), plus a human confirmation step consistent with human–AI interaction guidelines [11]. To our knowledge this validate-before-apply pattern has not been reported for network-education tools, whose mainstream representatives [12], [13], [14] offer no integrated conversational tutoring at all.

III. SYSTEM DESIGN

A. Architecture Overview

CAPA8 is a dependency-light web application (vanilla ES6 modules, no build step) with two surfaces that share one conversation store: a chat page for open tutoring and a topology simulator with an embedded tutor panel. The simulator supports ten device types (router, switch, PC, firewall, server, cloud, access point, and three industrial nodes: PLC, robot arm, AGV), link properties (latency, bandwidth, loss), a virtual terminal (ping, traceroute, ipconfig, ARP and routing inspection) animated over the graph, and eleven curated example topologies (LAN, VLAN routing, DMZ, campus, data center, industrial). A Node.js backend relays prompts to either a cloud provider or a locally hosted open-weight model, so institutions can run the entire stack on commodity hardware without sending student data to third parties.

Two design decisions tie the architecture to pedagogy. First, a static topology analyzer continuously detects eight classes of misconfiguration (duplicate IPs, isolated nodes, missing gateways, downed or lossy links, disconnected components) and renders them to the student; the identical issue list is serialized into the LLM context, so the tutor’s explanations refer to the same evidence the student can see. Second, conversation state persists across both surfaces, allowing a student to design a network in the chat, open it in the simulator, and continue the same dialogue.

B. Adaptive Scaffolding: the $Level \times Focus$ Matrix

Tutoring style is the product of a three-step *Level* and a two-value *Focus* (Table I), yielding six configurations that are compiled into the system prompt. Levels encode scaffolding intensity in the sense of graduated, fadeable support [6], [7]: the guided level forbids unexplained acronyms, requires analogies, warns about typical novice errors, and ends each turn with a reflection question; the technical level addresses the

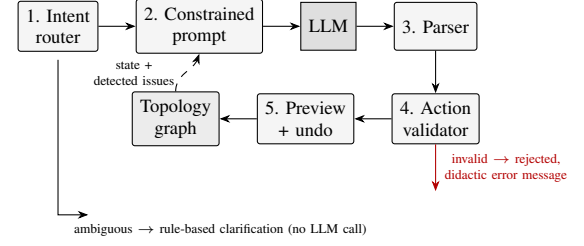


Fig. 1. The validate-before-apply pipeline. The LLM only proposes; deterministic stages decide what reaches the learner’s topology.

TABLE I
SCAFFOLDING CONFIGURATIONS ($LEVEL \times FOCUS$)

Level	Tutor behavior (applies to both foci)
Guided	Step-by-step, simple analogies, no unexplained acronyms, anticipates novice errors, closes with a reflection question.
Balanced	Direct and practical; at most three paragraphs or a concise list.
Technical	Dense, standards-referenced (RFC/IEEE), proactively flags design faults, peer register.
Focus	<i>Design</i> : propose and construct topologies. <i>Troubleshoot</i> : analyze the graph, explain faults, propose concrete fixes.

student as a peer and cites RFC/IEEE standards. Focus selects the task frame: *design* (constructive: propose and build topologies) or *troubleshoot* (diagnostic: locate and explain faults). Decoding temperature is additionally bound to intent—0.0 for modifications, 0.2 for conceptual explanation—so creative variability is spent on pedagogy, never on state changes.

C. Deterministic Intent Routing and Clarification

Before any model call, a rule-based classifier assigns each utterance one of six intents: *conceptual*, *graph query*, *diagnostic*, *clear modification*, *ambiguous modification*, or *full-graph application*. Classification uses imperative-verb lexicons, question patterns, and—characteristically—the live graph: a message is a *clear* modification only if it references existing node labels or a concrete target. Ambiguous requests (“add a device somewhere”) never reach the LLM; instead, a clarification dialogue is generated from the graph itself, offering the available attachment points as options. This keeps a class of interactions instant, free, and perfectly predictable, in the tradition of step-based ITS dialogue moves [8], and it determines downstream policy: which prompt blocks are injected, which temperature is used, and whether the action grammar is enabled at all.

D. Action Grammar and the Validation Layer

All state changes are expressed in a closed grammar of seven actions (add_node, add_link, update_node, delete_node, delete_link, set_link_status, apply_graph) serialized as JSON inside explicit [CAPA8_ACTION] delimiters. Three prompt-side mechanisms raise the probability that an action request actually yields well-formed blocks: a directive with paired wrong/correct counter-examples (e.g., gateway repair must

patch a node property, never add a cable), a seeded exchange of canonical examples injected only for action intents, and an automatic single retry with a stricter instruction when an expected action is missing.

What the model returns is still untrusted. A parser isolates the delimited blocks (malformed JSON is marked invalid and can never execute) and strips them from the text shown to the student. Each surviving action then passes semantic validation against the current graph: the action type must belong to the whitelist; referenced source and target nodes must exist; duplicate IP assignments and duplicate links are flagged; unknown device types are normalized through an alias table (`laptop` $\rightarrow$ `pc`, `internet` $\rightarrow$ `cloud`); and an `add_node` naming an existing node is rewritten into an `update_node`, converting the most frequent duplication hallucination into the action the model should have chosen. Rejected actions are not silently dropped: the validator emits an explanatory message in the chat (“source node *Router5* not found”), turning the failure into feedback for both the student and, via history, the model. During batch application, validation re-runs per action against the refreshed graph, so earlier actions in a batch cannot smuggle later ones past the checks.

E. Human-in-the-Loop Application

Validated actions still require consent at the right granularity. Batches of three or more actions open a preview panel where the student reviews and confirms the plan; any applied batch is one atomic undo step; and the frontend ignores all model-proposed canvas coordinates, retaining layout authority on the client. These choices follow established human–AI interaction guidance on efficient correction and graceful degradation [11]: the system makes the tutor’s agency visible, bounded, and reversible—properties we consider preconditions for trusting an acting tutor in a classroom.

IV. PRELIMINARY EVALUATION

A. Formal Verification of the Pipeline

The deterministic pipeline components are covered by 47 unit tests (validator, intent router, response parser, graph model), all passing. The validator tests certify the properties the reliability argument rests on: nonexistent references are rejected, unknown action and device types are rejected, alias normalization and the `add_node`-to-`update_node` rewrite behave as specified, and duplicate IPs raise warnings.

B. Reliability Benchmark

We probed the end-to-end pipeline with a scripted benchmark of 30 student-style utterances in Spanish against a fixed five-node topology seeded with deliberate faults (a PC without gateway, a PC without IP, an in-use IP address). The suite contains 16 clear modification requests (including traps: repairing the missing gateway, assigning the missing IP, referencing nonexistent devices, reusing an occupied IP), 6 diagnostic requests, and 8 conceptual or query utterances for which any emitted action is spurious. Every response was parsed and validated by the production code itself; in total 180 model interactions were evaluated.

TABLE II
FULL PIPELINE ACROSS DEPLOYMENT SCENARIOS (30 PROMPTS EACH)

Model	Deploy.	AER %	Spur. %	JSON %	Valid %	Interc.
Gemma 3 4B	Local	100	37.5	100	77.8	10
Gemma 3 12B	Local	100	0.0	100	66.7	12
Llama 3.3 70B	Cloud	100	0.0	100	80.0	11

Columns as in Table III. Local models served via Ollama on commodity hardware; the cloud model via Groq. Every model emits well-formed actions for all requests, yet none avoids semantically invalid ones, all intercepted before reaching the graph.

TABLE III
ABLATION OF PROMPT-SIDE MECHANISMS, GEMMA 3 4B LOCAL (30 PROMPTS PER CONDITION)

Condition	AER %	Spur. %	JSON %	Valid %	Interc.
Full pipeline	100	37.5	100	77.8	10
– seeded examples	100	37.5	100	92.0	4
– auto-retry	100	37.5	100	76.6	11
– system channel	100	37.5	100	68.9	14

AER: share of the 16 modification prompts answered with at least one parseable action. Spur.: share of the 8 conceptual/query prompts that wrongly emitted actions. JSON: well-formed blocks among all emitted blocks. Valid: parsed actions passing semantic validation. Interc.: invalid actions intercepted by the validator (would otherwise have corrupted the graph).

Across deployment scenarios: We ran the full pipeline under three scenarios spanning the privacy/cost spectrum a school faces: a small local model (Gemma 3 4B [15]), a larger local model (Gemma 3 12B), both served through Ollama on commodity hardware, and a cloud-hosted high-capacity model (Llama 3.3 70B via Groq). Table II reports the results. All three emit an action for every modification request and every emitted block is well-formed JSON, yet none is free of *semantically* invalid actions: 10 to 12 actions per model referenced a nonexistent node or duplicated an address and were intercepted before reaching the graph. Model capability changes *which* failure mode dominates—the 4B model wrongly emits actions on 37.5% of conceptual prompts, whereas neither larger model ever does—but it does not remove the need for validation: even the 70B cloud model produced eleven structurally invalid actions, all caught. Interception is by construction independent of model quality.

Ablation of the prompt-side mechanisms: To isolate each prompt-side mechanism we ablated them on the smallest and hardest case, Gemma 3 4B, where structured-output reliability is most fragile: the full pipeline, no seeded examples, no auto-retry, and the directive moved from the system channel into the user prompt (Table III). Routing the directive through the system channel is the dominant aid—removing it raises intercepted actions from 10 to 14 and drops semantic validity to 68.9%. The seeded examples and auto-retry show no clear benefit at this model size (the model already emits an action for every request, so the retry never fires), a candid result we report rather than overstate. The validator’s interception nevertheless holds across all four conditions.

The trap items show the division of labor between prompt-side steering and validation: the duplicate-IP and nonexistent-device traps were caught by the validator (as a warning and as didactic rejections, respectively), whereas the gateway and IP-repair traps were answered with the correct property-patch

action rather than the spurious-cable shortcut—an outcome attributable to the counter-example prompt blocks, not to validation.

C. Illustrative Scenario

A student loads the faulty LAN and asks, in troubleshoot focus and guided level, why PC1 cannot reach another subnet. The analyzer has already flagged the empty gateway and that flag travels inside the model context; the tutor explains gateways with an analogy and proposes a one-action fix patching PC1.gateway. Had it proposed a cable to a nonexistent “Router5” instead, the validator would have rejected it with an explanatory message, and nothing would have touched the student’s network. The student applies the fix (one undo step), reruns ping in the virtual terminal, and watches the packet animation succeed.

V. DISCUSSION AND LIMITATIONS

The validation layer guarantees *structural* soundness, not pedagogical optimality. The gateway trap delineates the boundary precisely: connecting PC1 to the router with a new cable is structurally valid yet didactically wrong; CAPA8 currently steers the model away from it with counter-examples and analyzer hints, but the validator alone would not catch it. Encoding such domain-pedagogical rules (e.g., “gateway problems are property problems”) as first-class validations is a natural next step. The benchmark, while reproducible and run through the production code path, covers three models, one topology, and Spanish utterances only; it measures reliability, not learning. No claims about learning outcomes are made: the evaluation of comprehension gains, trust calibration, and scaffolding-level effects requires the classroom study outlined below. Finally, we believe the validate-before-apply pattern transfers directly to other artifact-centric subjects—electronic circuits, database schemas, chemical process diagrams—wherever an LLM tutor is allowed to touch the thing the student is building.

VI. CONCLUSION AND FUTURE WORK

CAPA8 demonstrates that an LLM tutor can be given hands—the ability to construct and repair the learner’s network topology—without inheriting the model’s unreliability, by interposing deterministic intent routing, a closed action grammar, state-aware semantic validation, and human-confirmable application between proposal and effect. Preliminary technical evidence (47 passing unit tests; a 180-interaction benchmark spanning local 4B and 12B models and a 70B cloud model) indicates that the validation layer intercepts every structurally invalid action regardless of model capability, that larger models reduce spurious emissions but never eliminate the invalid actions the validator must catch, and that the architecture runs on hardware schools already own. The next step is a controlled classroom study with networking students (pre/post concept inventory, usability, and scaffolding-level comparison), to be conducted under institutional ethics review. This paper reports no human-subjects data.

ACKNOWLEDGMENT

AI use disclosure (per IEEE and TALE policy): portions of this manuscript were drafted and edited with the assistance of a generative AI system (Claude, Anthropic), used for text composition, benchmark scripting, and LaTeX formatting under the direction of the authors, who reviewed, verified, and take full responsibility for all content, code, and results. The CAPA8 system itself was designed and implemented by the authors. [Names, affiliations, funding, and the public code repository are withheld for double-blind review and will be restored upon acceptance.]

REFERENCES

- [1] E. Kasneci *et al.*, “ChatGPT for good? On opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, p. 102274, 2023.
- [2] P. Denny *et al.*, “Computing education in the era of generative AI,” *Communications of the ACM*, vol. 67, no. 2, pp. 56–67, 2024.
- [3] M. Liffiton, B. Sheese, J. Savelka, and P. Denny, “CodeHelp: Using large language models with guardrails for scalable support in programming classes,” in *Proc. 23rd Koli Calling Int. Conf. on Computing Education Research*, 2023, pp. 1–11.
- [4] Z. Ji *et al.*, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [5] L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA: Harvard University Press, 1978.
- [6] D. Wood, J. S. Bruner, and G. Ross, “The role of tutoring in problem solving,” *Journal of Child Psychology and Psychiatry*, vol. 17, no. 2, pp. 89–100, 1976.
- [7] B. J. Reiser, “Scaffolding complex learning: The mechanisms of structuring and problematizing student work,” *Journal of the Learning Sciences*, vol. 13, no. 3, pp. 273–304, 2004.
- [8] K. VanLehn, “The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems,” *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.
- [9] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, “NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” in *Proc. EMNLP 2023: System Demonstrations*, 2023, pp. 431–445.
- [10] G. Poesia *et al.*, “SynchroMesh: Reliable code generation from pre-trained language models,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2022.
- [11] S. Amershi *et al.*, “Guidelines for human–AI interaction,” in *Proc. CHI Conf. on Human Factors in Computing Systems*, 2019, pp. 1–13.
- [12] Cisco Networking Academy, “Cisco Packet Tracer,” 2024. [Online]. Available: <https://www.netacad.com/courses/packet-tracer>
- [13] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: Rapid prototyping for software-defined networks,” in *Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*, 2010, pp. 1–6.
- [14] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken, NJ: Pearson, 2021.
- [15] Gemma Team, Google DeepMind, “Gemma 3 technical report,” *arXiv preprint arXiv:2503.19786*, 2025.